

Imitation Learning, Action Chunking

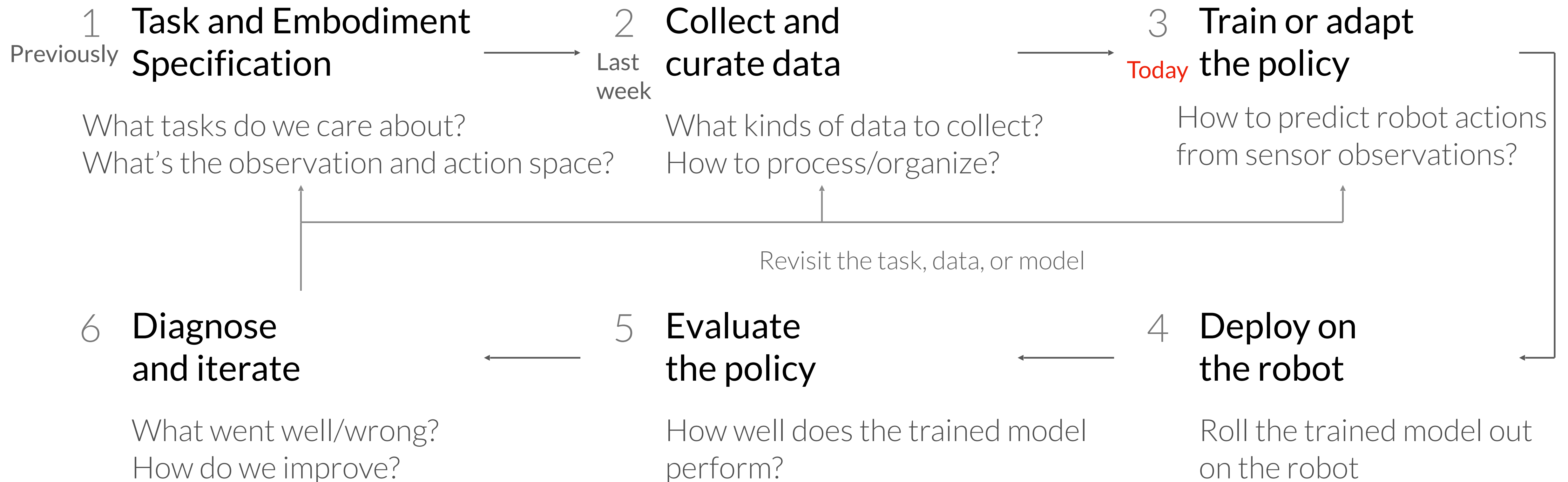
EN.601.[4|6]98 Hands-on Robot Learning

Krishna Murthy Jatavallabhula

Administrivia

- (Likely) next week: in-class robot assembly (assignment 0)
 - Still awaiting final piece of shipment 🙌 🙌 🙌
- Teams announced last week. Please meet-and-greet teammates
- New CA onboard: welcome Yinzhe (Steven)!
- Look out for assembly instructions and prerequisites post early next week
 - Gist: you will need to bring a laptop to class that's setup with LeRobot (suffices to have 1 laptop per team)

The *modern* robot learning lifecycle

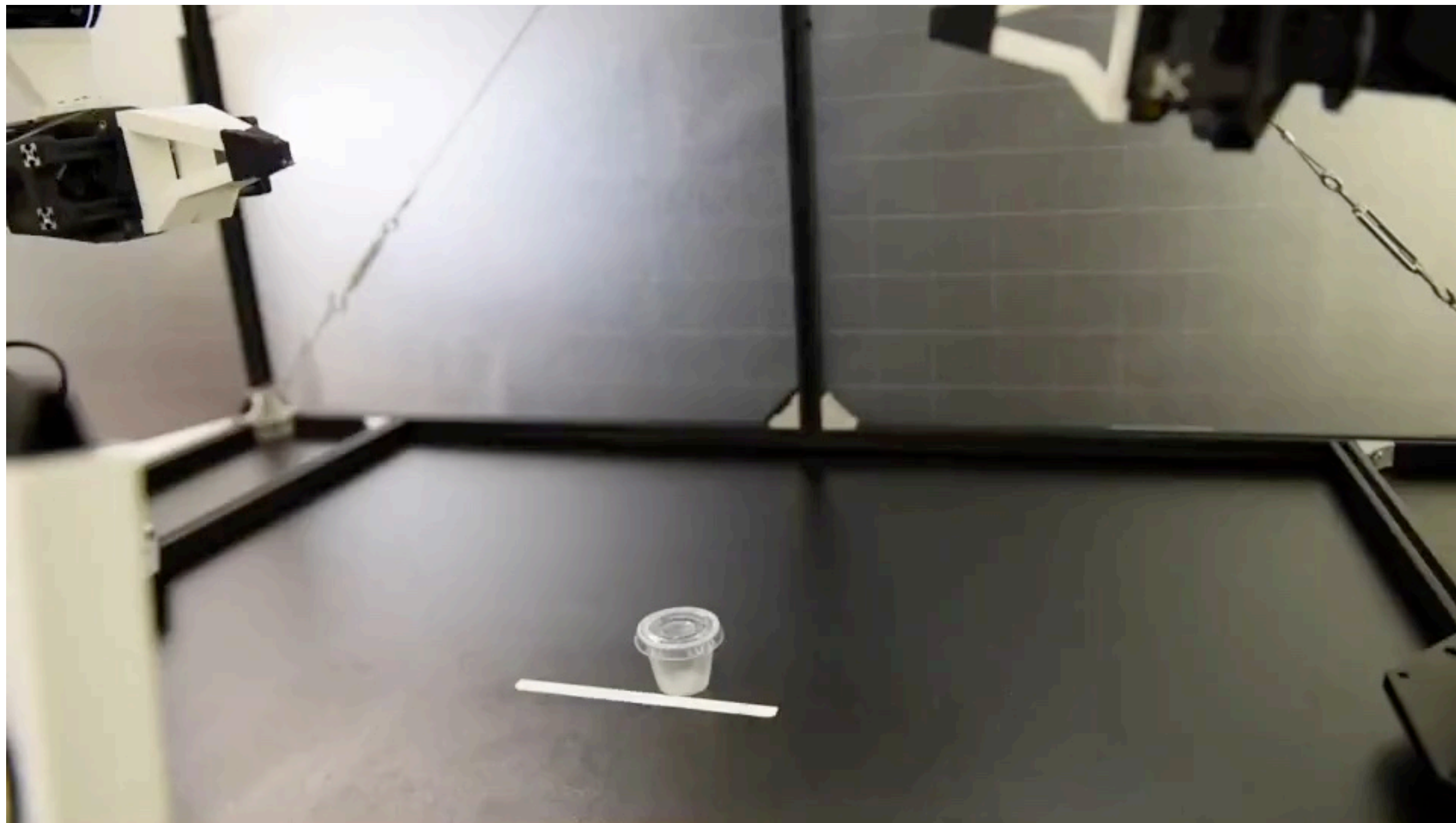


Today's focus: Single-task imitation learning

- So far, we've looked at
 - How to design/build a robot
 - (you'll soon have built your team's SO-101)
 - How to collect data / demonstrations for robot learning
- Today, we'll look into how to take the collected data and learn to do a single task autonomously
 - single-task: robot does just the one task, defined implicitly via the collection of demonstrations
 - the learned behavior is often referred to as a *policy*
 - specifically, we will consider *imitation learned* policies (i.e., behaviors that mimic/replicate demonstrations)
- a.k.a. behavioral cloning (and under the hood, it's basically supervised learning)

By the end of the lecture, you'll be able to

- Understand the terms policy, state, observation, action, Markov decision process (MDP), behavioral cloning, distribution/covariate shift, causal confusion, action chunking, ...
- Appreciate what action chunking does, and understand how it is still limiting
- Understand the design choice/space of behavioral cloning methods in general, and tradeoffs therein
- (Next lecture) ~~Take away a framework that lets you train and debug policies (a.k.a. 'gold-standard policy debugging')~~
- Analyze how low-level controller behavior and feedback influences policy rollouts
- Explain why adding more information may sometimes be counterproductive



Formalisms / notation

- t -> time (usually discrete in this course, unless noted otherwise)
- $\mathbf{s}_t$ -> state
 - variables, if known, fully specify the configuration of the robot + external world
 - *almost* never true in practice (e.g., you almost never precisely know what's behind you)
- $\mathbf{o}_t$ -> observation
 - the part of the state that is *observed* (e.g., images are an observation of the underlying state of the world)
- $\mathbf{a}_t$ -> action
 - an action that the robot takes (e.g., joint positions + gripper open/close)
- $\pi(a_t | o_{0:t})$ -> policy
 - determines what action to take at the next timestep based on one or more previous observations
 - in this course, we will largely use $\pi(a_t | o_t)$

Formalism: Markov Decision Process

- Markov assumption (a.k.a. the *state* is Markovian)
 - $P(s_{t+1} \mid s_0, a_0, \dots, s_t, a_t) = P(s_{t+1} \mid s_t, a_t)$
 - i.e., if you know the *current state*, no other piece of past information is needed to estimate the next state (except for actions you will take from the current state)
- A fully-observable, finite-horizon Markov Decision Process (MDP) is defined by
 - $S, A \rightarrow$ state space, action space
 - $P(s_{t+1} \mid s_t, a_t) \rightarrow$ state transition function
 - $R(s, a), \rho_0, T \rightarrow$ reward function, initial state distribution, time horizon
- Not covered in this class: partially-observable MDPs, a.k.a. POMDPs (most robotics problems are POMDPs)

Behavioral cloning: learning setup

- Assume we have collected *demonstrations*: a sequence of observations (camera images, joint positions) and corresponding actions (joint positions, gripper open/close/width)
 - Recall: we need to use *commanded*, not realized/measured joint poses
- Given this, we *learn/train* model that predicts the action the demonstrator (a.k.a. the *expert*) would take, given the same observation
 - This is done via supervised learning
- Once trained, the learned model takes robot observations, predicts actions, and these actions are then executed on the robot. This results in new observations, which result in newer action predictions, and so on...
 - Note: this assumes that the low-level controller is near-identical during teleop data collection and inference!

Behavioral cloning: training objective

$$\min_{\theta} \frac{1}{N} \sum_{i=1}^N \|f_{\theta}(o_i) - a_i\|_2^2$$

- The model f_{θ} (usually a neural net) assumes we're tackling an MDP, so we split up each demonstration into (observation, action) tuples
- Each (observation, action) tuple gives us one training example of the form (input, label)
- θ denote the neural network parameters that need to be learned
- The training process involves shuffling the dataset, presenting a small subset of data (N samples at a time, also known as a *mini-batch*) to the model, and optimizing the model parameters using variants of stochastic gradient descent
- For continuous actions, commonly used *loss function* is mean squared error (shown in above equation)
- Alternatively, one can predict a distribution over actions and maximize the likelihood of demonstrated actions
- Important (and *untrue*) assumption above: all training examples are i.i.d. (independent and identically distributed)

Distribution / covariate shift in behavioral cloning

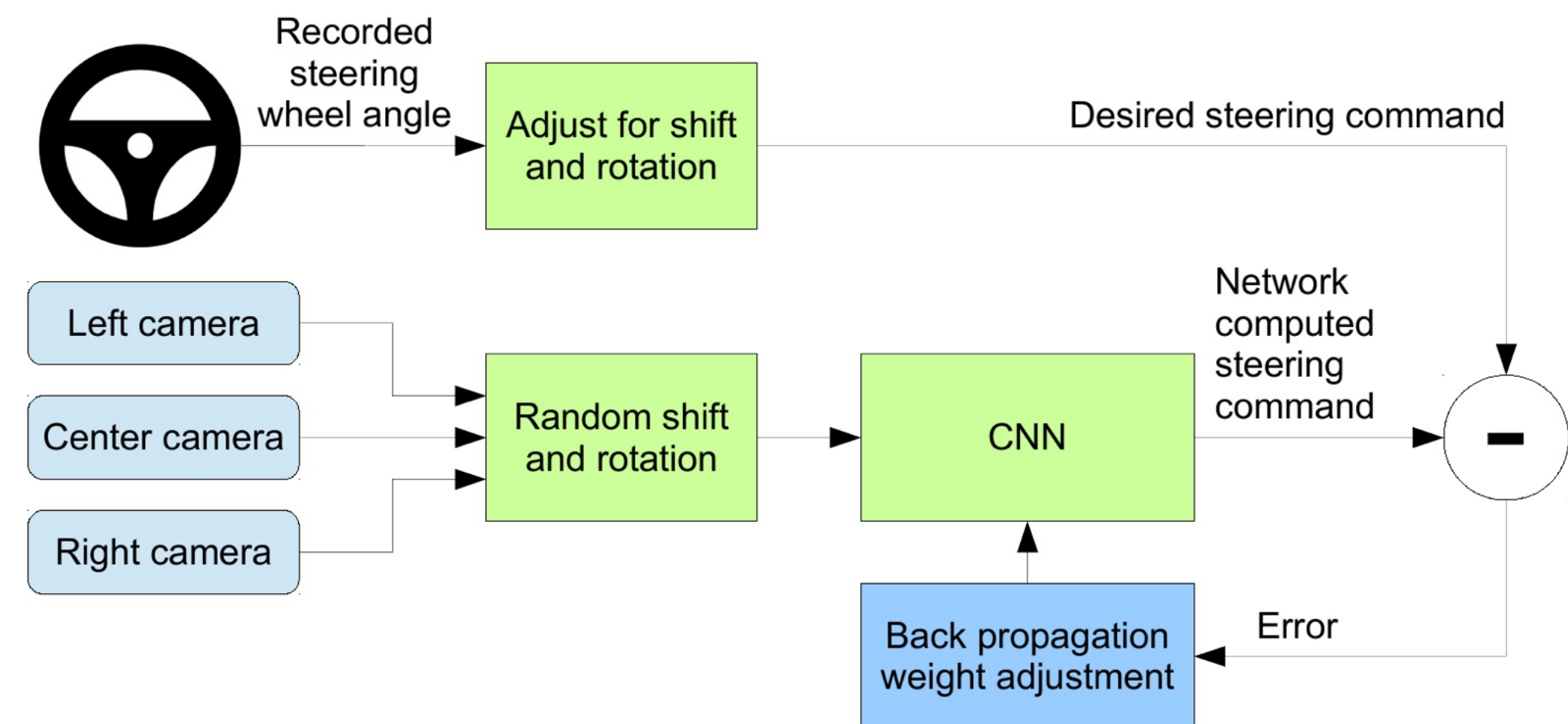
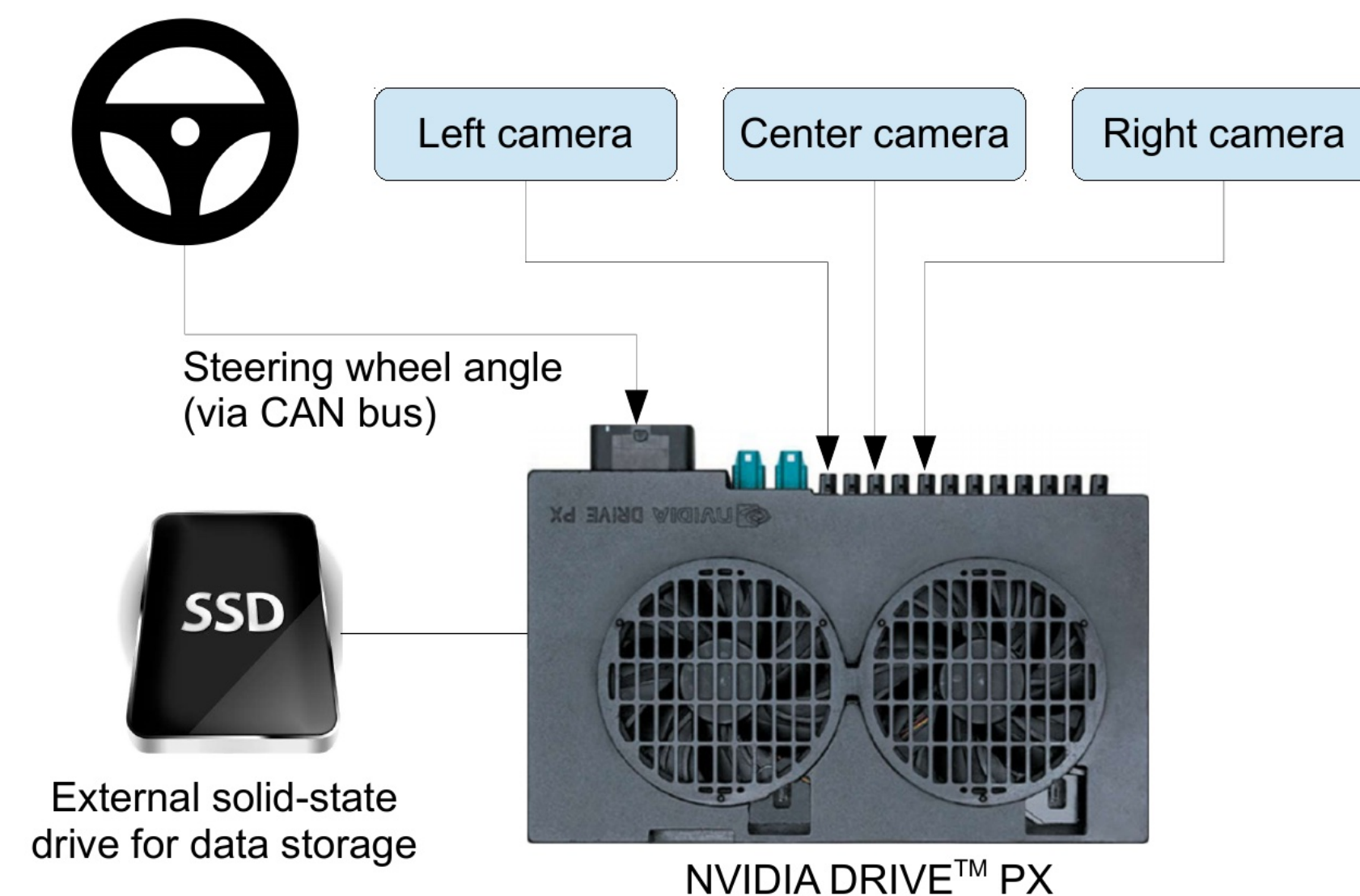
- Distribution shift => the inputs encountered during deployment are different from those seen at train time
 - Some of these changes are external to the policy: lighting, camera placement, object appearance, etc.
 - Others come from the policy itself: the policy makes an error and the resulting actions move the subsequent observations (and thereby actions) away from demonstrated trajectories
- The latter is particularly important in imitation learning
 - Even with the same robot and the same initial scene, a learned policy can cause a distribution shift in observations, leading to erroneous action prediction
- Demonstration distribution needs to be well thought-out:
 - Extremely clean demonstration trajectories (e.g., generated by a planner/controller with privileged info) are generally worse-off than noisy, teleoperate trajectories
 - More demos along the same trajectory will improve action prediction accuracy, but come at the cost of reducing variance in the training distribution
- Need to carefully consider the distribution of states/observations in the demonstration distribution vs those a policy visits during deployment

Compounding errors (because we train assuming iid, but execute sequentially)

- Assume we execute a policy for T time steps
 - Also, suppose that the policy makes an error with probability at most ϵ at each step (w.r.t. expert actions)
- Assume deterministic dynamics (again *untrue* in practice). Until the first mistake, the learner follows the expert trajectory. The probability that a mistake has been made by time t is at most $(t - 1)\epsilon$
- In the worst case, once a mistake is made, a robot never recovers. It keeps making a mistake every subsequent time step, and the extra expected cost is given by
 - $\epsilon [0 + 1 + 2 + \dots + (T - 1)]$, which is $\frac{T(T - 1)}{2}\epsilon$, or $O(T^2\epsilon)$
- This is a worst-case bound, and assumes robot doesn't recover
- This also doesn't take into account any instantaneous costs associated with a mistake (e.g., dropping a fragile object)

Case study: recovery / corrective examples in behavioral cloning for driving

- Human driver drives a car, and images + steering commands are logged
- Policy takes in an image and predicts the next steering command
- If trained only on centered driving, policy doesn't have enough data to inform what to do once the car drifts away from center
- Idea: collect corrective steering examples by adding cameras to the left and right sides of the center camera, generating *virtual* observations (i.e., what would the car potentially see if it were off center)
- Use the augmented dataset to train the policy



Case study contd. (low-level details matter more than you'd think)

- Action is parameterized as $(1 / r)$, where r is turning radius (this makes it vehicle-independent)
- Recorded video observations are sampled at 10 fps for training (prevents successive images from being 'too similar')
- Simulate a number of additional shifts (not just fixed left/right off-center shifts) by synthesizing images for simulated off-center shifts (use traditional, planar homography-based methods, leveraging points on the road plane)
- Data is tagged with lighting, weather conditions, driver activity (and the metadata was useful in curating the final dataset used for training)
- Human drivers may not be driving in the center of the lane all the time (i.e., they make less mistakes, but also lack high precision) — this needs to be accounted for during eval

Aside: demonstration distributions, adding noise during data collection

- The above approach augmented data *procedurally*, using predefined left/right cameras (plus some augmentation). What if deployment results in shifts that aren't adequately captured by this setup?
- DART perturbs the action executed when collecting demos
 - disturbance pushes robot into states that require expert to correct
 - record the expert's *commanded* action as supervision
- The resulting demos contain local recovery behavior around nominal trajectories
- The amount and structure of the noise matter
 - Too little disturbance => not enough additional coverage.
 - Too much disturbance => can produce states from which the task cannot be completed
- This only changes data distribution; which in-turn impacts behaviors of policies trained on the data

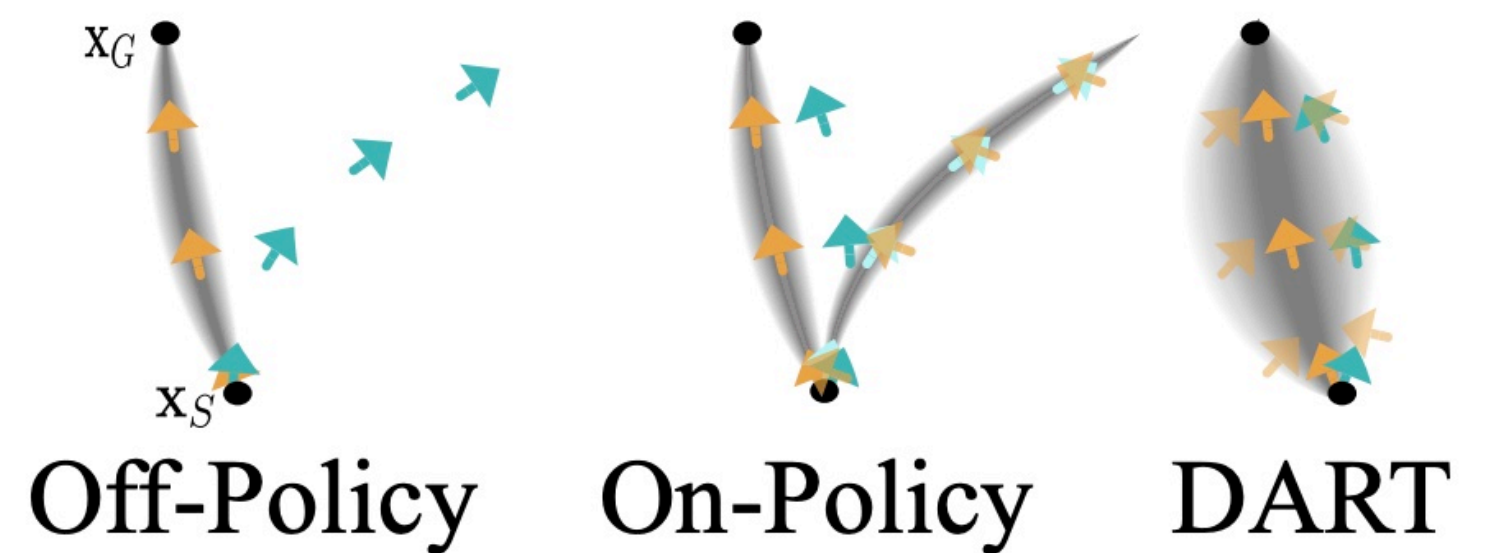


Figure 1: Robot Learning to reach a goal state x_G . The grey denotes the distribution over trajectories. Left: Off-Policy learning in which the supervisor, the orange arrows, provides demonstrations. The robot, the teal arrows, deviates from the distributions and incurs high error. Middle: On-Policy which samples from the current robot's policy, the light teal arrows, to receive corrective examples from the supervisor. Right: DART, which injects noise to widen the supervisor's distribution and provides corrective examples. DART is off-policy but robust.

DAgger: Collecting corrections where the policy needs them

- Another approach: deploy the policy, and ask the expert what the policy should have done in the situations it actually visits
- DAgger (Dataset Aggregation):
 - Train initial policy with expert demonstrations
 - Rollout the trained policy and obtain expert action labels on the visited states
 - Add these examples to the dataset and retrain
 - Repeat
- Dataset now includes observations induced by the learner, rather than only observations induced by the expert
- Dataset also includes a mix of on-policy and off-policy trajectories
- Practical challenges:
 - expert must now be involved at deployment time, in addition to data collection
 - may need multiple rounds of iteration, and still not have full state coverage / high performance
 - expert must provide useful corrections, incl. in awkward/unsafe states (e.g., moments before a collision)

Evaluating behavioral cloning policies (and why it's hard)

- Recall two assumptions from earlier: training data is i.i.d. ; the problem is an MDP
- Both are untrue ~all the time in practice
- In practice, we execute the policy sequentially — policy gets observations, predicts actions, these actions are deployed on a robot, and the loop repeats
- This is no longer iid (every subsequent observation depends on the previous action)
- Most real-world robotics problems are not MDPs (e.g., task on the right isn't, if all we get is an image observation)
- Evaluation in robotics (and robot learning in particular) is hard



Task: *pick up the marker and place it on its left*

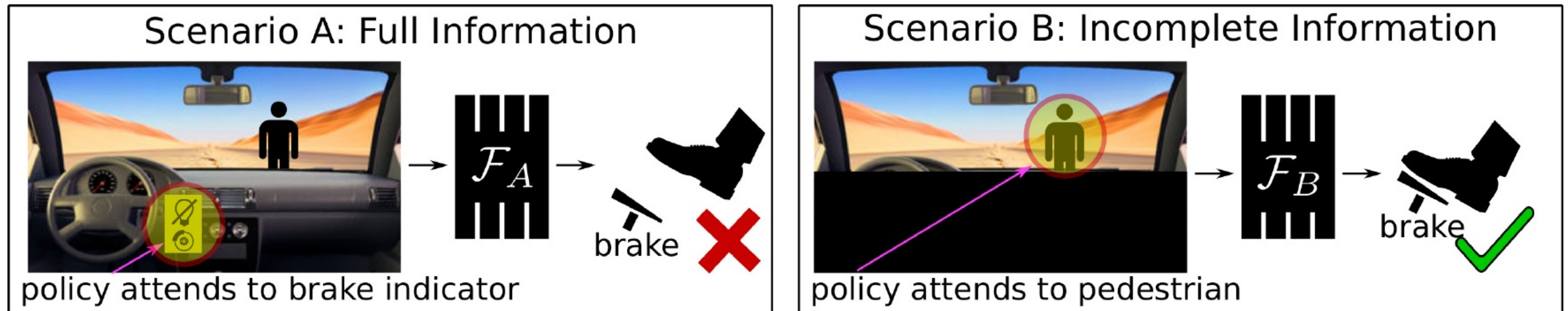
Q: yet-to-start the task? or completed-the-task? What state does the image indicate?

(Image observation does not fully uniquely specify state)

Evaluating behavioral cloning policies (and why it's hard)

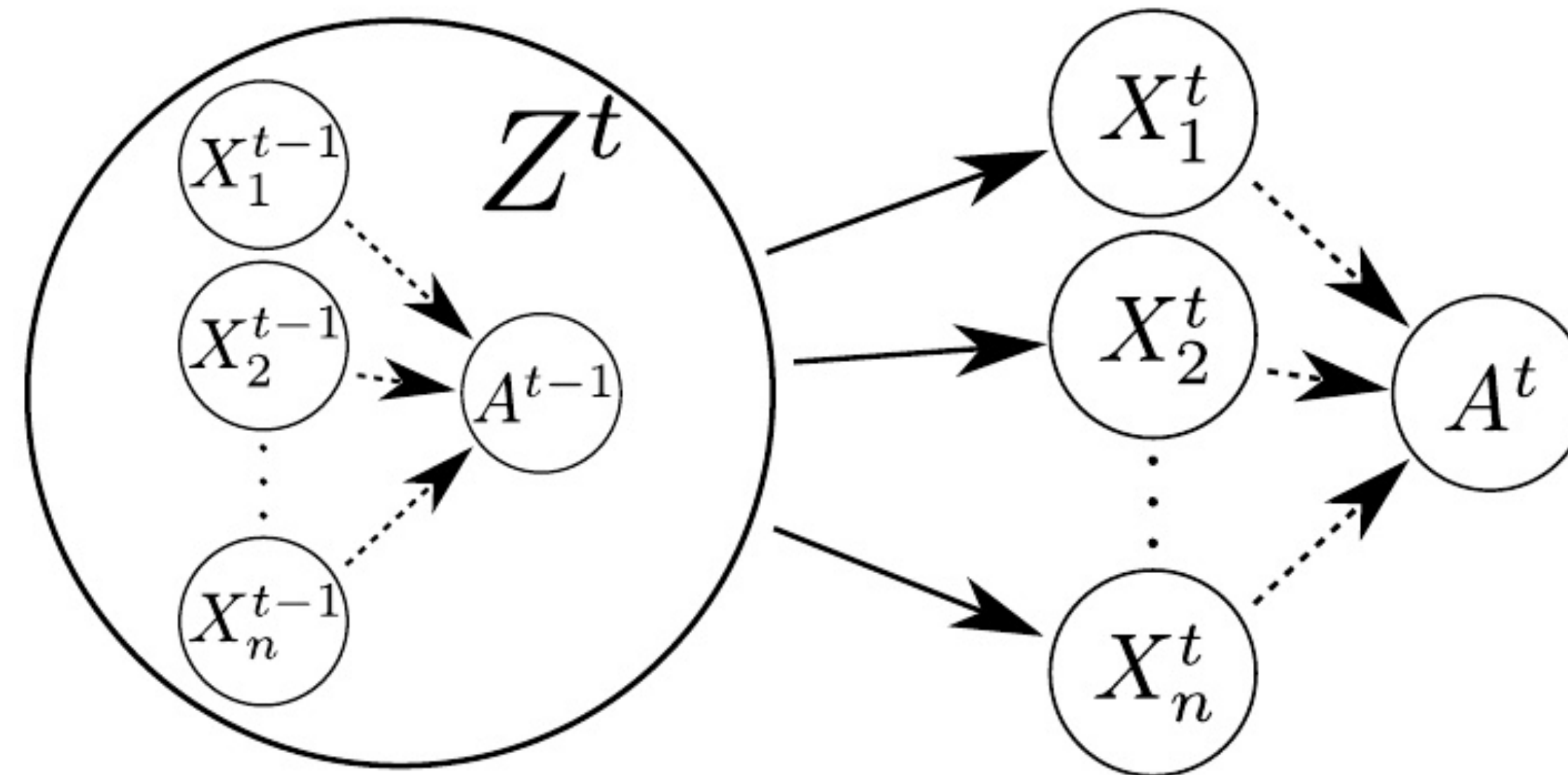
- Strategy 1 (batch statistics)
 - Compute loss (e.g., mean squared error) on a held-out val/test set (just as in supervised learning)
 - This tells you how the policy would do in an i.i.d. world (which is untrue in practice)
- Strategy 2 (replay a sequence)
 - Replay a train/val episode, and note all points where divergence occurs
 - This is similar to the first strategy, except that we're emulating sequential execution (while an oracle 'resets' to the correct action)
 - Informs us of states / action dimensions where the trained policy is subpar
- Strategy 3 (evaluate in simulation)
 - Test the policy in a simulated environment
 - Works only when there's a replicable env in sim, with little-to-no visual / physics misalignment
 - Hard to do in practice (but, maybe not anymore, h/t GPT Astra?), but a very valuable testbed for anything safety critical (e.g., surgery) or hard-to-evaluate in real-world (e.g., space robots)
- Strategy 4 (deploy on real robot)
 - Test the policy on the real robot
 - Closest signal you can get on policy performance
 - Downsides: real evals take time + effort (e.g., environment resets); this results in small sample sizes that can't be extrapolated to meaningful performance metrics across all states/observations
- In practice, progress through each of these strategies in phases

Causal confusion (adding more information can degrade policy performance)



- Suppose the observations contain cues that are highly correlated with the expert behavior
 - For example, a brake light / indicator can indicate that the expert is braking
 - Model (incorrectly) establishes a causal connection that it should brake (only when) brake light is on
 - When deployed, policy never uses brake (because brake indicator is never on by default)
- It learns this incorrect causal association because predicting actions from a cue leads to low MSE loss
- i.e., safeguards are needed to prevent learning of incorrect causal relationships among observations and actions

Causal confusion: correlated demonstrations



- Correlation $\neq$ Causation
- Passive demonstrations may not tell us which relationship will remain useful when we intervene by deploying a new policy
- For those interested, read this paper (TL;DR identify relevant causal inputs by using interventions and policy evaluation)

Why does naive BC fail often for robot manipulation?

- Policy ends up in states / observations missing from the demonstrations
 - How to fix: recovery data, interactive labeling, dataset augmentation
- Observation may not contain enough information to determine the next action
 - History, additional sensing, or task context can help *some* of the time (i.e., if the policy uses the info appropriately)
- Policy can rely on (causally incorrect) correlations - i.e., improving supervised prediction can hurt closed-loop behavior
- One weird fix that unlocked a lot of progress in BC / IL for robot manipulation (and what we'll focus on a majority of the lecture): *action chunking*

Action chunking

a.k.a. predicting and executing a sequence ('chunk') of actions



What action should the robot take, based on just the image?

Assume gripper actions are continuous (i.e., policy predicts gripper target width in mm)

Predicting an action chunk

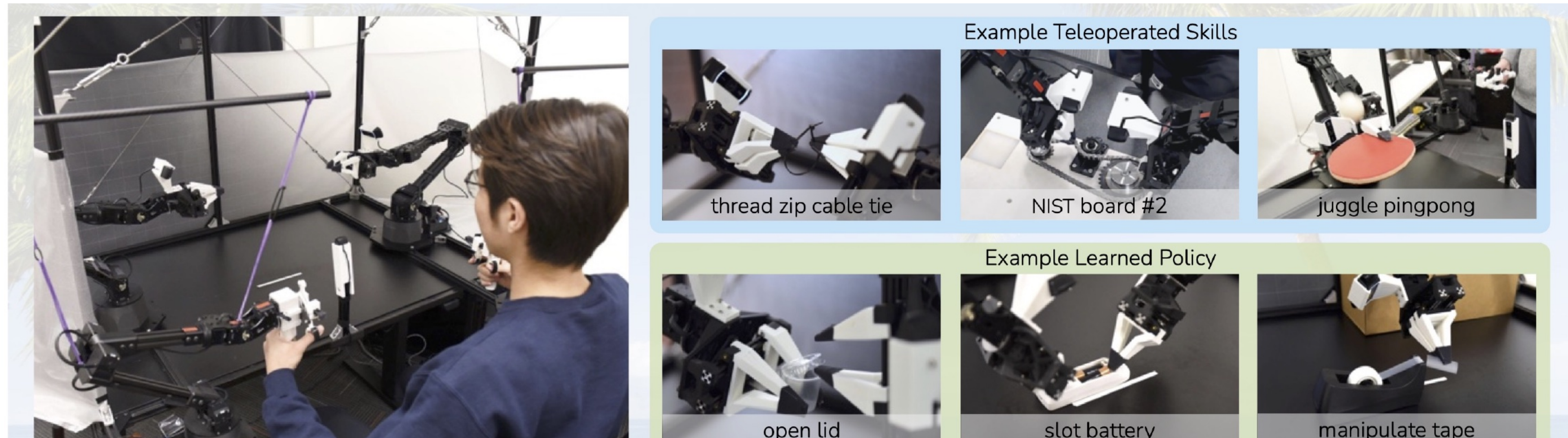
$$\hat{A}_t = f_{\theta}(o_t) = [\hat{a}_t, \hat{a}_{t+1}, \dots, \hat{a}_{t+H_p-1}]$$

- Instead of predicting one command, which can be ambiguous, or quite hard to learn from an image, predict the next H_p commands jointly
 - This can still be trained using supervised learning objectives (the predicted vector just has more dimensions now)
- For the example on the previous slide, the predicted sequence can now contain multiple gripper commands for the same input image (because we can predict a sequence of close commands)
- Multiple actions can be executed, before querying a new image and predicting a new set of actions
- This helps (partly) get around the Markov assumption — policy (+ the deployed controller/harness) gives the *illusion* of memory
- Works well for *quasi-static* tasks (i.e., tasks where you can ‘pause to think’ and the world doesn’t change)
- Action chunks also allow you another axis of control: policy deployment / execution

Observation, prediction, and execution horizons

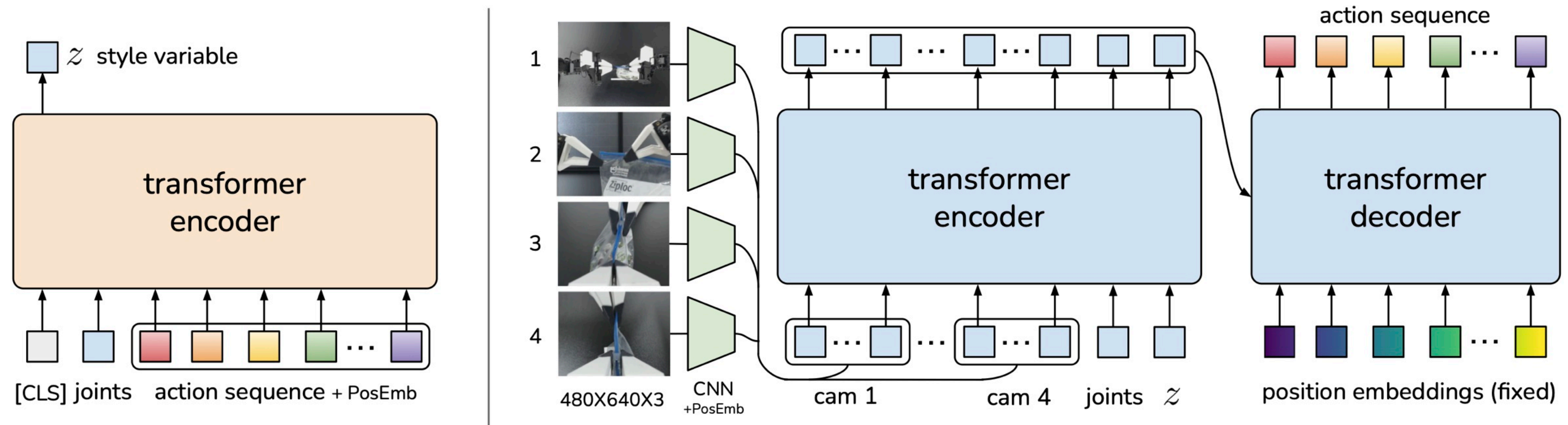
Nomenclature	What this means	In this class (i.e., SO-101 w/ LeRobot)
Observation horizon	How many time steps in the past the camera observes (i.e., number of current + past images + poses)	One current image from each camera + measured joints
Prediction horizon (H_p)	Number of future actions predicted in one query (also called <i>chunk size</i>)	30 commands (1 sec — a widely used default by many labs)
Execution horizon (H_e)	Number of predicted actions that get executed before re-querying the policy	10 commands (but this is really TBD by you)
Control frequency	Rate at which commands are issued to the robot	30 Hz: predict 1 s, execute about 0.33 s

Action Chunking with Transformers (ACT)



- The first (of many) imitation learned approaches we'll learn in this course
- A great candidate policy for Assignment 2 (single-task policy learning)
- ACT combines action-sequence prediction with a transformer (and other optional modules)
- Teleop data is collected with ALOHA, a low-cost bimanual teleoperation system

ACT: policy architecture



- Images from multiple cameras are *tokenized* and fed alongside proprioception. Model optionally receives a latent variable z encoding history of poses
- A transformer decoder uses one query per future timestep to predict the entire action chunk in parallel
- Use commanded actions (i.e., leader poses) as prediction targets
 - difference between leader and follower poses implicitly determines amount of force applied in ALOHA teleop

ACT: Observation history (Optional C-VAE)

- In a set of demonstrations, the same task can be performed in different ways
 - Demos could differ in approach trajectory, timing, or contact sequence
- During training, ACT optionally trains a conditional variational autoencoder (C-VAE) module that receives the current joint positions and the demonstrated action chunk
- C-VAE predicts a distribution over a latent variable z that summarizes variation in the demonstrated behavior
- Decoder then predicts the action chunk using the latent (z), image tokens, and current joint positions
- A KL-divergence term regularizes the encoded distribution toward a standard Gaussian prior (regular (C-)VAE training)
- Encoder has access to future actions only at training time (these are not needed once the C-VAE is trained)

ACT: the training loss

$$\mathcal{L} = \underbrace{\mathbb{E}_{z \sim q_\phi(z|q_t, A_t)} \left[\|A_t - \hat{A}_t\|_1 \right]}_{\substack{\text{Reconstruction loss} \\ \text{Match the predicted action chunk} \\ \text{to the demonstrated action chunk (L1)}}} + \underbrace{\beta D_{\text{KL}}(q_\phi(z | q_t, A_t) \parallel \mathcal{N}(0, I))}_{\substack{\text{KL regularization} \\ \text{Keep the encoder distribution close} \\ \text{to the standard normal prior}}}$$

Regularization weight

Expectation: average over z sampled from the training encoder

- The reconstruction term asks the predicted chunk to match the demonstrated commands
 - The original released implementation uses L1 reconstruction loss
- The KL term limits how much demonstration-specific information the encoder can place in z
 - Its weight β changes the balance between reconstruction and latent regularization
- Near the end of an episode, pad incomplete action chunks and mask the padded entries in the reconstruction loss
- Masking, normalization, and loss reduction need to agree with the implementation you train
- In practice, you can safely drop the C-VAE (KL divergence term) and train just the action chunk predictor (I'd highly encourage you to take that path for Assignment 2, esp. if you haven't worked with real robots before)

ACT: Inference

- The original ACT deployment sets $z = 0$ and omits the C-VAE during policy inference
- Policy receives current camera images and measured joint positions, and predicts an action chunk
- Execution loop will take a prefix (subset of first H_e actions) from that chunk, execute them, and then observe again and query the policy
 - Original ACT also uses *temporal ensembling* (on a later slide)
- Setting $z = 0$ makes this deployment deterministic for a fixed observation
 - Using a CVAE training objective therefore does not, by itself, mean we are sampling diverse behaviors on the robot. For this, it has to also be used during inference.
- Take-home message: every ACT implementation is different because they'll use different data / observations / CVAE on/off / and prediction/execution horizons (i.e., in assignments, you shouldn't expect code you downloaded from the internet to work directly on your robot — you'll need to think through the full-stack to debug/analyze)

How ACT policy learning workflow will look like in practice

- This is the bare-minimum you'll need to do for assignment 2 this class.
- Hypothetical task: Pick up a red cube and place it in a tray
- We first determine what the observations and actions are
 - Example: two camera views, five measured arm joint positions and the gripper position
 - Ordering of the joints, absolute-vs-relative position predictions, gripper action parameterization — all of this matters
- Collect demonstration data (typically using leader-follower teleop)
 - Ensure demonstration consistency and quality
 - Recall discussion earlier this lecture on recovery behavior and noise
 - Demo quality often trumps quantity (you'll still need to determine how many demos you'll need)
- If needed, tune the controller. Log controller metadata.
- Follow our 'gold-standard debugging' process outlined in a later lecture

Practical ACT (and other) policy training gotchas

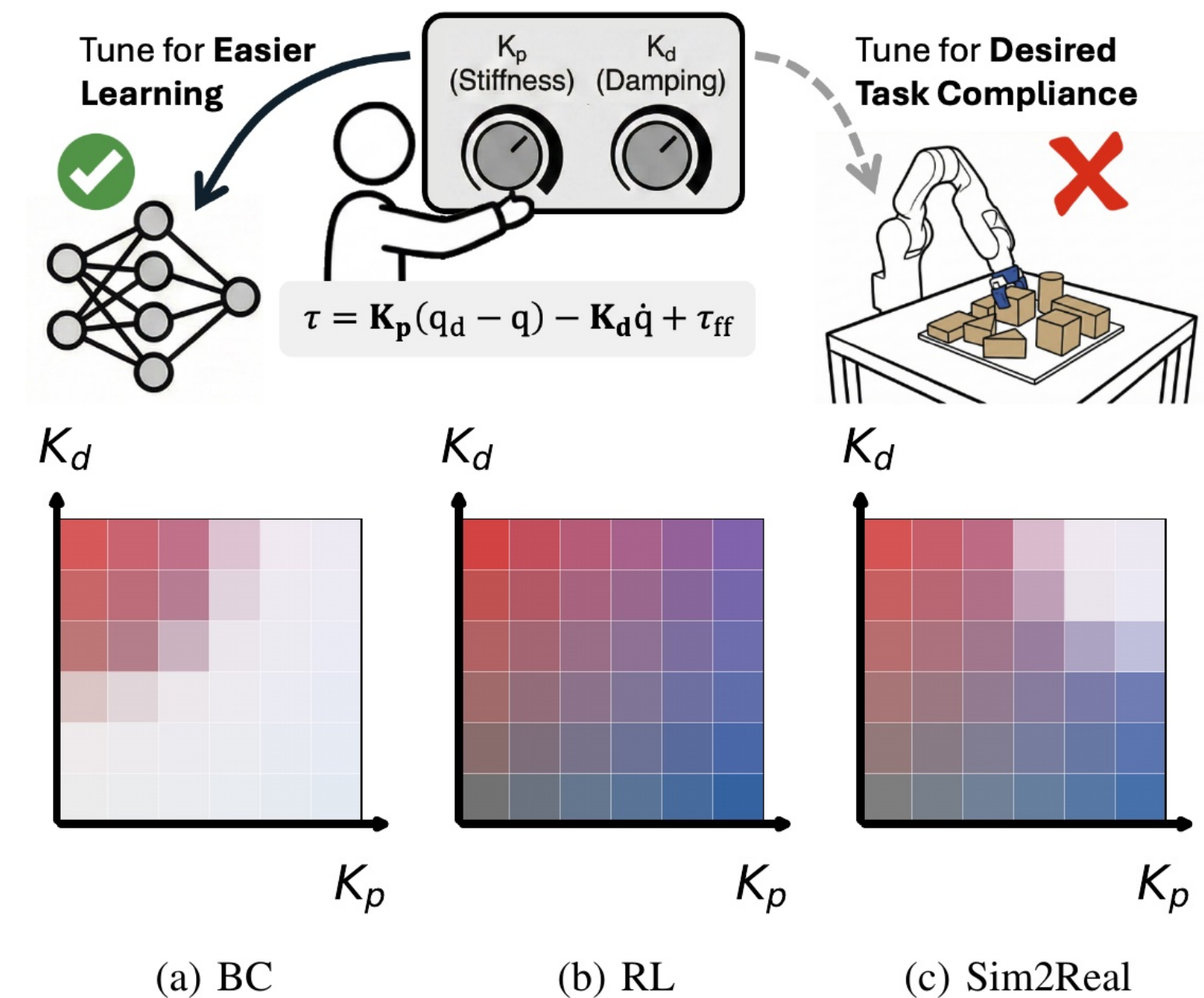
- Preprocess dataset to remove large swaths of 'zero-actions' (e.g., at the beginning of a demo, when the robot is at a 'home' pose) -> else you get policies that don't want to move
- Induce variation in initial configurations -> gives the policy much-needed variance in training demonstrations
- Normalize data (and pay extra care on what dims are normalized and how). Gripper dimensions need to be treated with extra, extra care -> often the difference between 'policy doesn't work' vs 'policy works quite well'
- Validate controller settings across train and deployment. If you've removed and re-installed a motor or a part, recalibrate the robot
- Execute a training demonstration on the robot hardware encoded into training data format and decoded w/ the inference controller to ensure you can 'replay' trajectories consistently
- Start with a small subset of high-quality / clean episodes and overfit to that episode
- Do not move the robot/camera once you start collecting training data, until you finish deploying/evaluating the policy
- Try and maintain consistent background and illumination throughout

Action chunking: upsides + downsides

- Predicting action chunks enables models to reason/represent temporal relationships b/w actions
 - e.g., sequences of joint angles that result in a grasp / approach / retreat / etc.
- In addition to consistency, chunking reduces the number of queries to a policy (useful when policies incur long inferences times or are offboard)
- But, this also means the policy doesn't always act on the most recent observations
 - E.g., if the policy knocks off an object, or if a pick fails, execution will continue for a few 100s of milliseconds before the policy gets a new observation and predicts a new action chunk
- The policy doesn't directly account for low-level controller errors
 - Assumes that controller tracks predicted joint positions perfectly
 - Implicitly assumes low-level controller is consistent across training and validation

Aside: controller gains affect imitation learning

- Low-level controller determines how commanded (position) targets are realized during both collection and deployment
- Changing controller gains changes the trajectories, errors, and corrective behaviors represented in the demonstration distribution
- Stiff controllers can track position targets closely, but result in harder-to-handle dynamics for learned policies
- Main finding: compliant, over damped controllers aid BC



(Aside) Why does action chunking work?

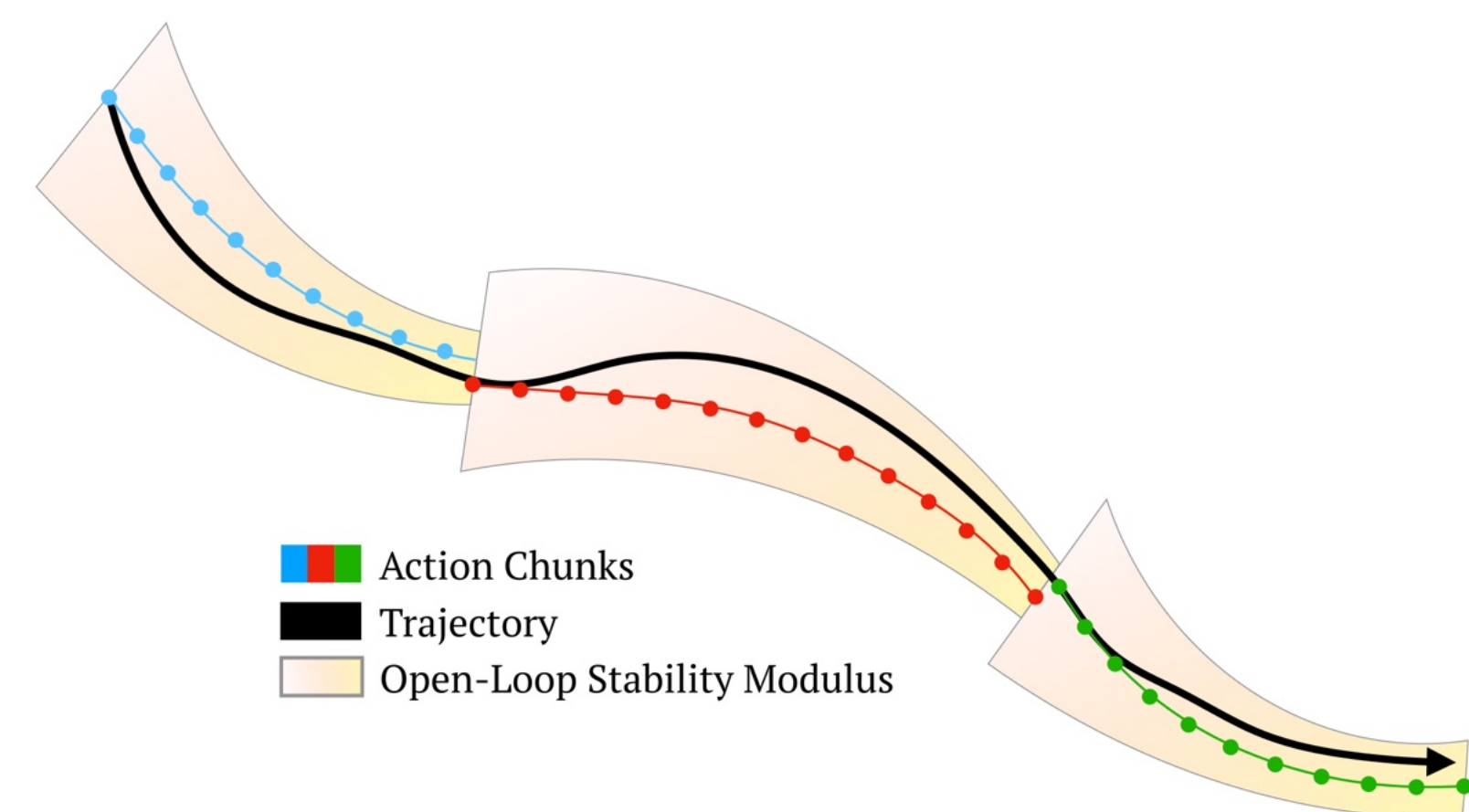
TL;DR. We don't know (yet). There's a couple reasonable explanations though

Explanation 1: Lower train/val error doesn't lead to lower deployment error

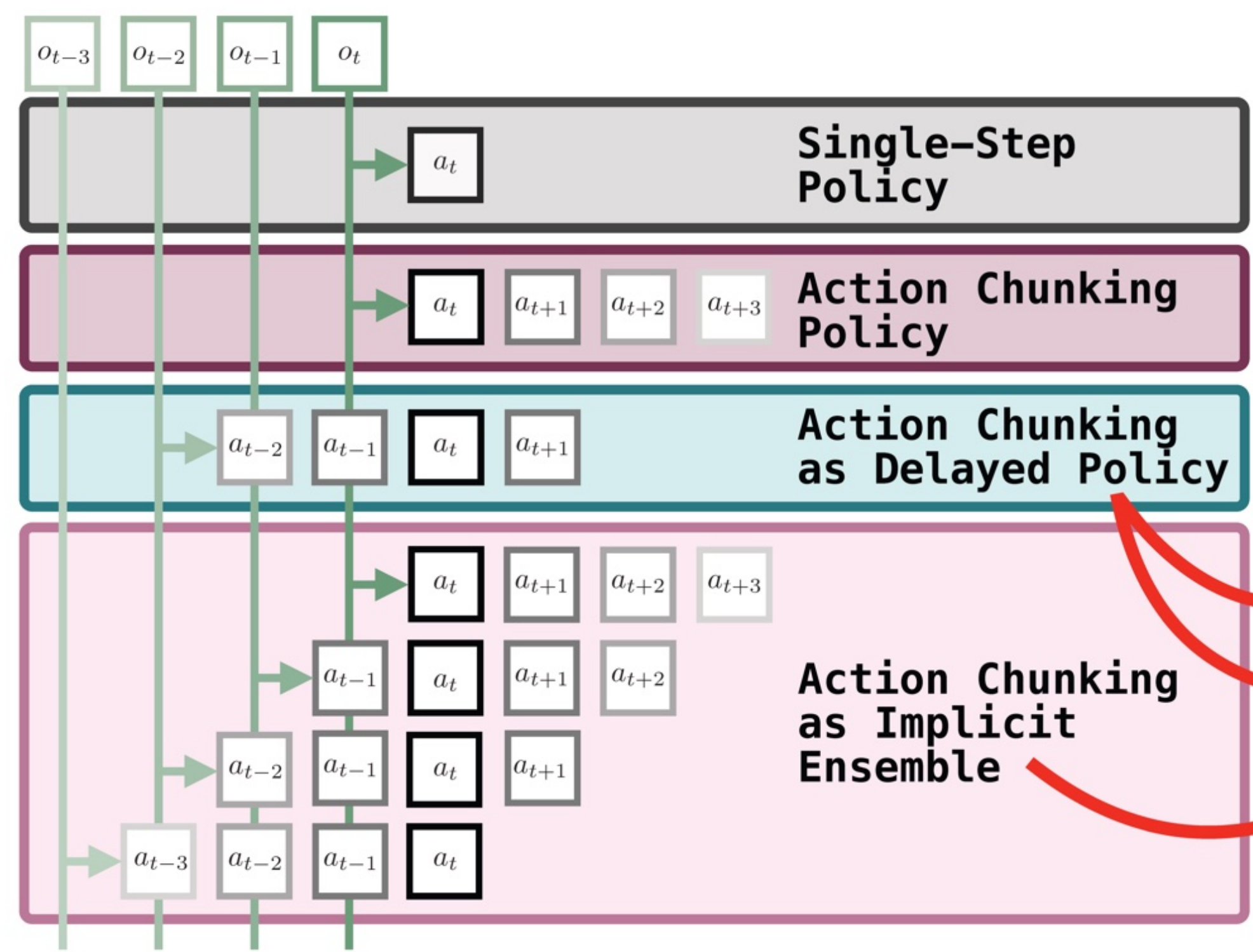
- Simchowitz, et al. analyze the failures of imitation learning in continuous control settings
- They design settings where good action prediction on expert data does not give a good deployed policy
 - Policy errors interact with the system's dynamics and move it away from the demonstration distribution
- Recall limitations inherent in BC from earlier in the lecture (demos don't capture all the ways in which policy output should change in response to a disturbance)
- Simchowitz et al. show that some BC policies are more stable than others (chunked policies are comparatively more stable)

Explanation 2: Action chunking leads to more stable dynamics

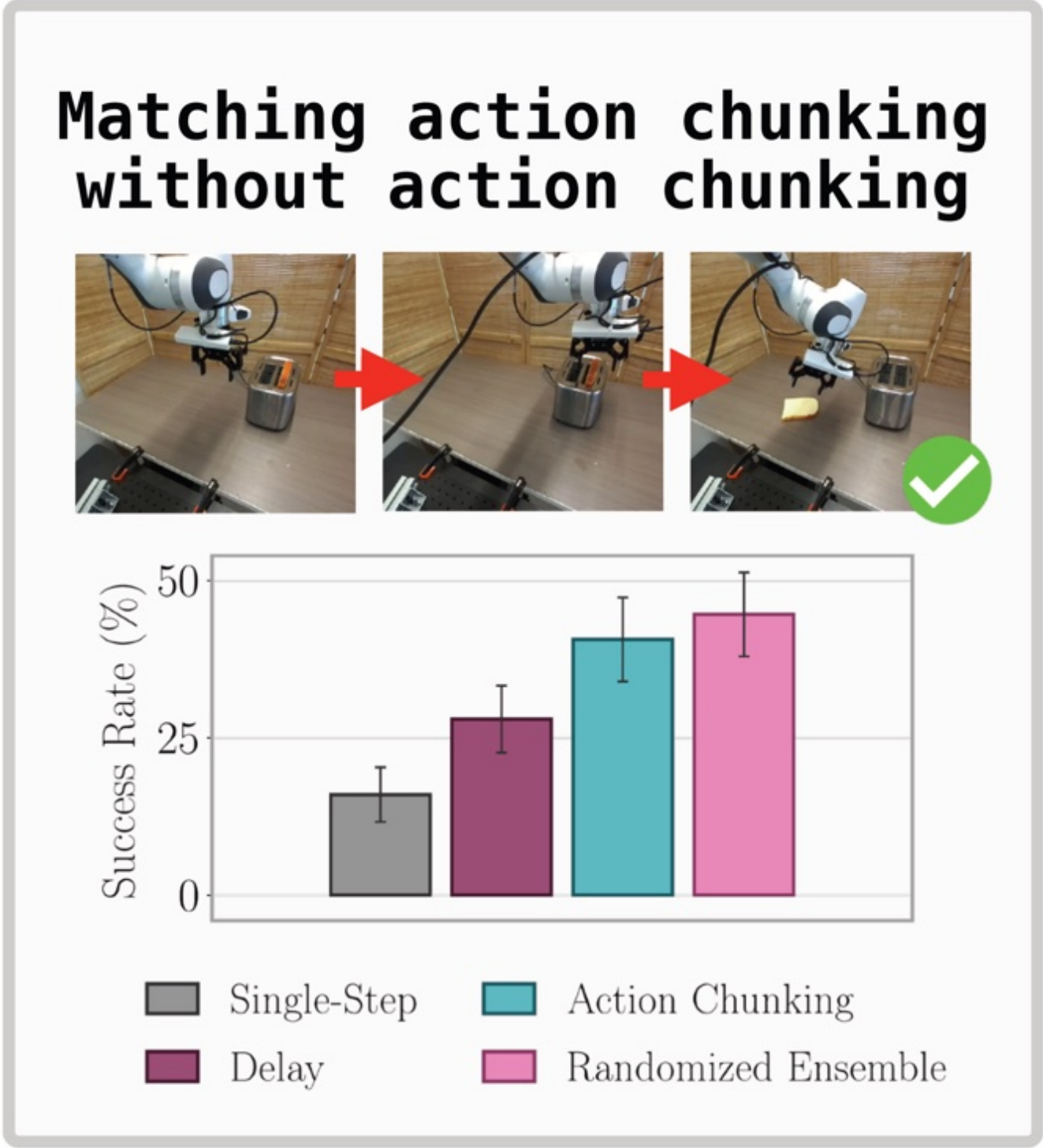
- Executing a chunk reduces the number of queries to the policy (which can help with error reduction in general)
- If policy is re-queried for every single action, one incorrect action (or low-level controller error) can possibly impact everything that follows
- However, in executing action chunks, we can mitigate the rate at which error accumulates
- (Under appropriate assumptions) Zhang et al. show that predicting and executing chunks can reduce error propagation effects



Explanation 3: (Empirically) Chunking helps w/ Non-Markovian expressivity, etc.



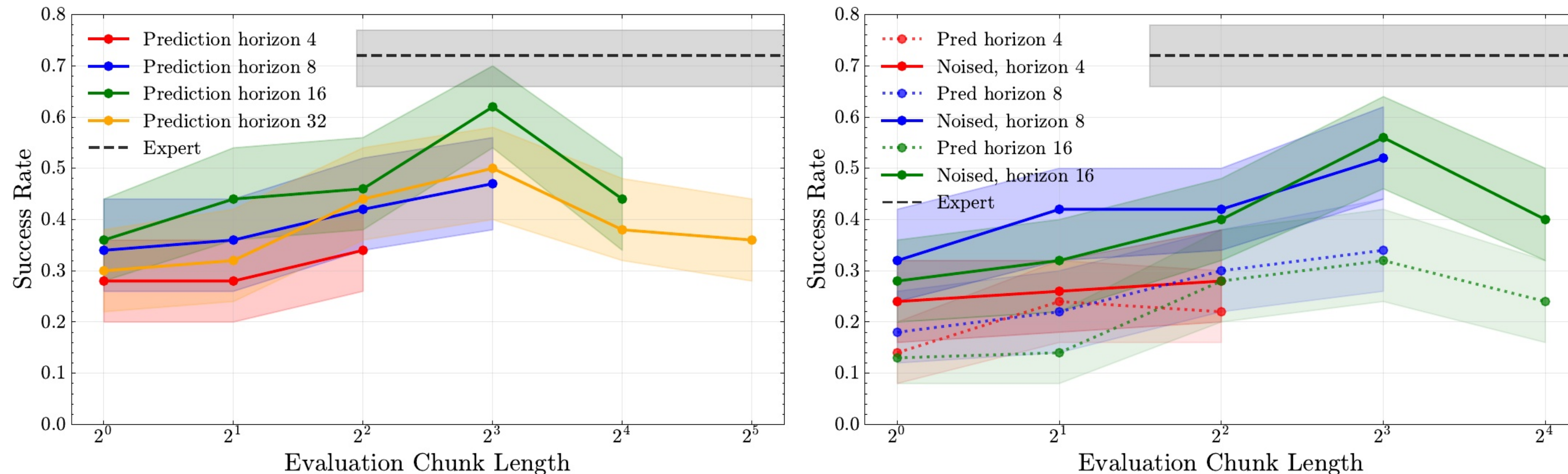
- ### Why does action chunking improve BC performance?
- ✗ Temporal Consistency
 - ✗ Horizon Reduction
 - ✗ Representation Learning
 - ✓ Non-Markovian Expressivity
 - ✓ Reduced Compounding Error
 - ✓ Ensemble Generalization



(Aside) More action chunking musings

Impact of prediction and execution horizon lengths, Temporal ensembling, and added context

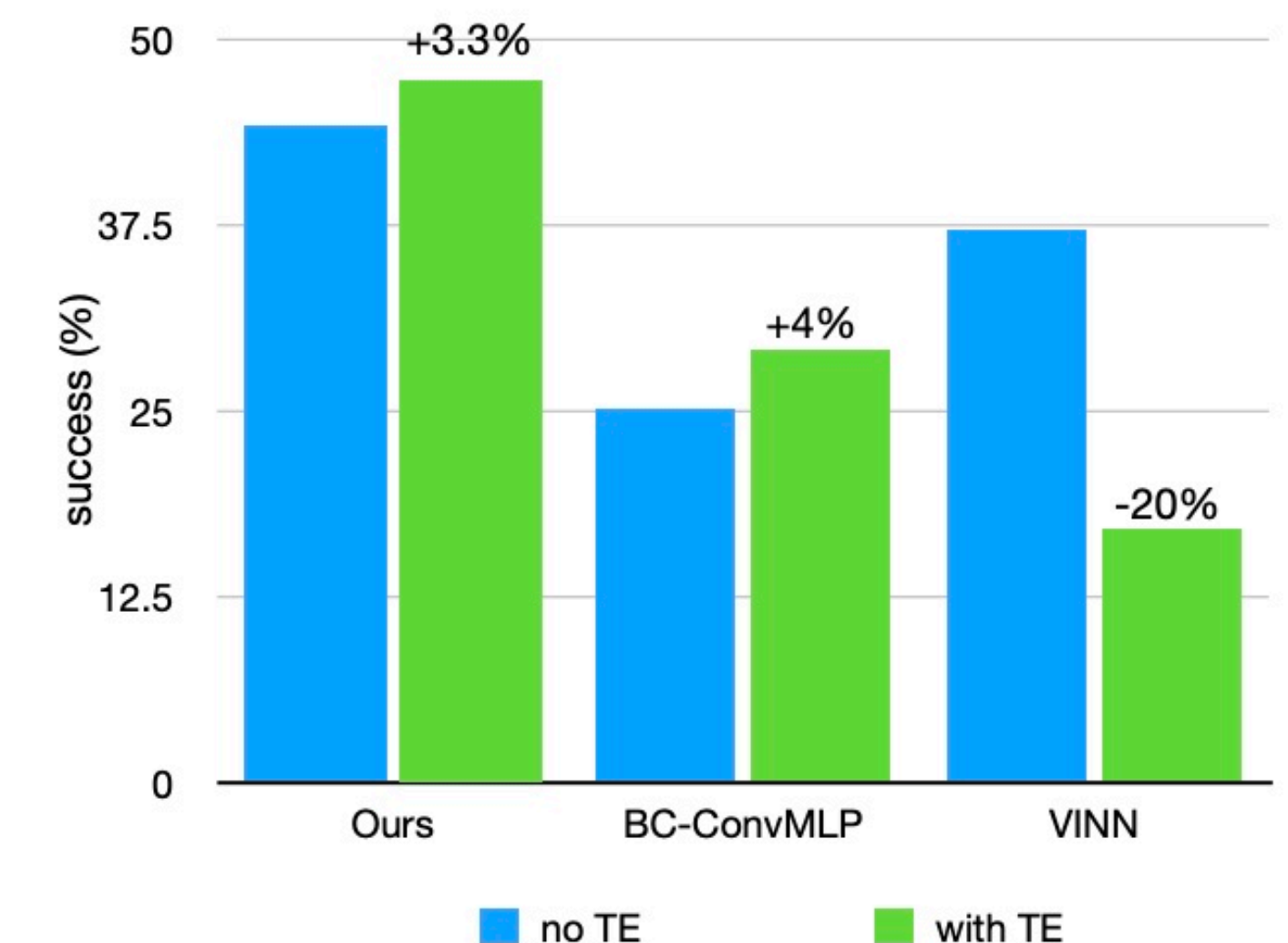
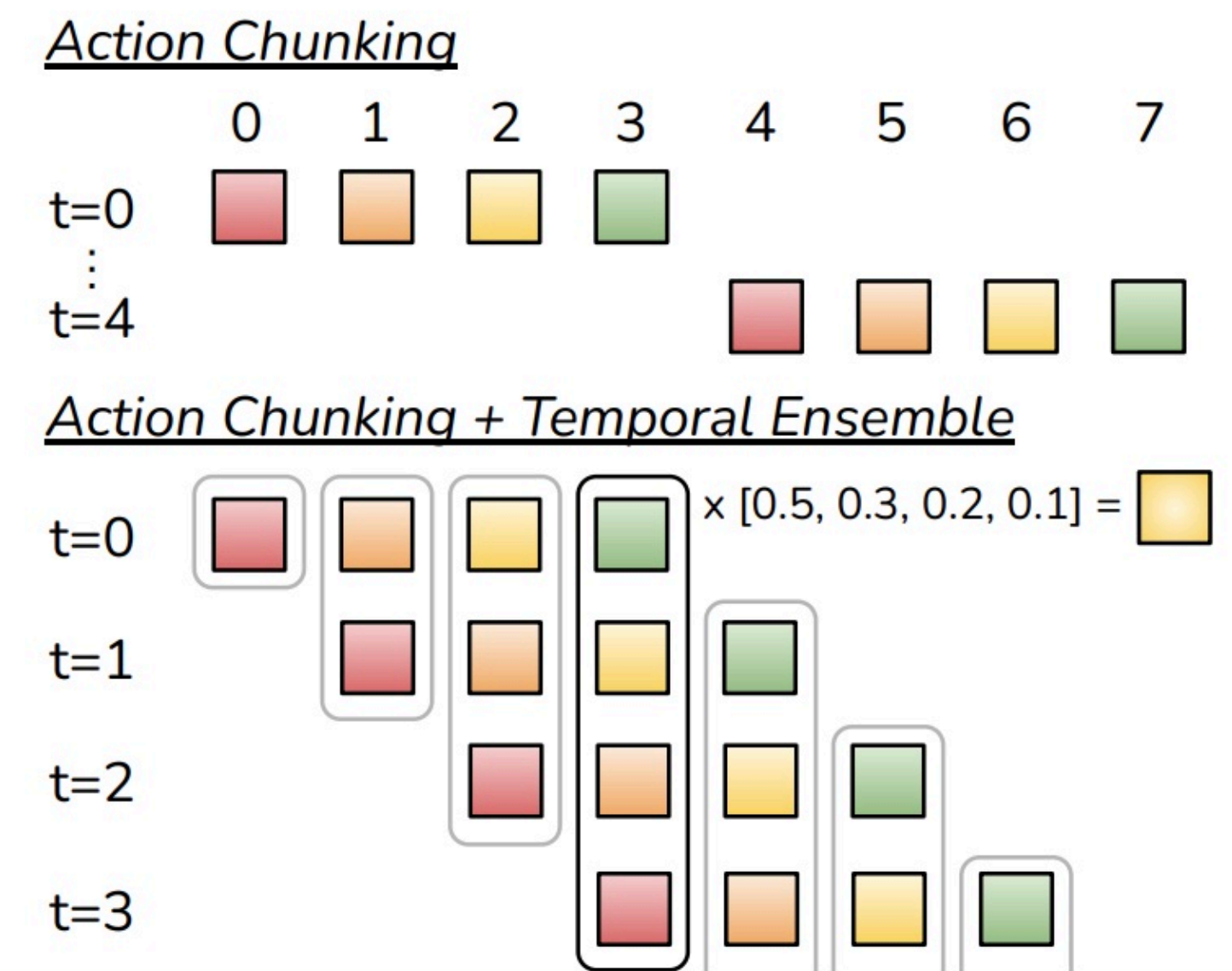
Musing 1: Predicting more actions, executing more actions



- In this tool-hang experiment, Zhang et al. analyze the impact of the prediction horizon and the evaluation horizon used
- Success improves over an intermediate range of execution lengths, then falls for some settings
- Left plot: policies trained on 100 expert trajectories
- Right plot: 50 trajectories, with and without noise injected during collection

Musing 2: Temporal Ensembling

- Another way to mitigate noise is by averaging predictions over multiple predictions
- Referred to as ‘temporal ensembling’ (TE)
- The original ACT experiments claim ensembling to be beneficial, but in general, jury’s mixed
- My guess: whether or not TE helps depends greatly on the data distribution
- When the demonstration distribution is multimodal, TE can quite literally average across actions from multiple *modes*, resulting in movements that don’t solve the task at all
- Similar observations when demonstration distribution contains trajectories with multiple speeds, and TE averages across them



Musing 3: More context doesn't always improve reactive policies

- Zeng et al. revisit open-loop action chunking and the role of policy context
- Long execution horizons primarily help short-context policies imitate non-Markovian experts
- With suitable context and enough data, reactive policies improve (in some settings)
- However, while context can resolve missing information, but it can also make newer shortcuts available (recall causal confusion)
- So, there isn't a universal chunk size / execution horizon



Imitation learning and Action chunking: Food for thought

- How does policy know when a task ends? (Does a policy know when a task ends?) What does it do after?
 - What'd happen if you let the policy run for a minute after completion?
- How do we ensure our task is Markovian in an action chunking framework?
 - Action chunk prediction horizon, execution horizon
- How do we know if our policy will work well in-practice?
 - Validation loss, Closed-loop trajectory prediction, hardware-in-the-loop tests
 - Next lecture: 'gold-standard readiness test' to determine whether a policy is ready to be evaluated on an SO-101 robot
- What kinds of data do we collect to train our policies?
 - How do low-level details (e.g., controllers) impact the data distribution?
- How do we move away from deterministic policies to stochastic ones (e.g., ones that produce multiple possible actions for each observation?)
 - next 'lecture' (generative policies)

Questions?